

JavaScript Spread Operator (...)

A Complete Beginner-Friendly Guide

Core JavaScript Concepts · Easy English · With Examples

1. What is the Spread Operator?

The **spread operator** is three dots written as **...** It **expands** (spreads out) an array, object, or string into individual pieces.

Think of it like opening a gift bag and laying all items on a table:

```
const bag = ['pen', 'book', 'ruler'];

// Without spread – you pass the whole bag:

console.log(bag);           // ['pen', 'book', 'ruler']

// With spread – items come out one by one:

console.log(...bag);        // pen book ruler
```

■ *The spread operator (...) and the rest operator (...) look the same but do opposite things. Spread EXPANDS values; rest COLLECTS them.*

2. Spread with Arrays

2.1 Copy an Array

Spread makes a **new copy** of an array. Changing the copy does not affect the original.

```
const original = [1, 2, 3];

const copy     = [...original];

copy.push(4);

console.log(original); // [1, 2, 3] – unchanged
console.log(copy);     // [1, 2, 3, 4]
```

■ *Without spread, doing `const copy = original` makes both variables point to the SAME array — so changes in one affect the other.*

2.2 Merge / Combine Arrays

You can join two or more arrays into one using spread.

```
const fruits = ['Apple', 'Mango'];

const veggies = ['Carrot', 'Spinach'];

const all = [...fruits, ...veggies];

console.log(all);

// ['Apple', 'Mango', 'Carrot', 'Spinach']
```

2.3 Add Items While Spreading

You can insert new items anywhere while combining arrays.

```
const nums = [2, 3, 4];

const more = [1, ...nums, 5, 6];

console.log(more); // [1, 2, 3, 4, 5, 6]
```

2.4 Pass Array Items as Function Arguments

Some functions (like `Math.max`) need individual values, not an array. Spread solves this perfectly.

```
const scores = [45, 92, 78, 61, 88];

// OLD way (before spread):

Math.max.apply(null, scores); // 92

// NEW way with spread:

Math.max(...scores); // 92 – much cleaner!

Math.min(...scores); // 45
```

2.5 Convert a String to an Array of Characters

Strings are also iterable, so spread works on them too.

```
const word = 'Hello';

const chars = [...word];

console.log(chars);

// ['H', 'e', 'l', 'l', 'o']
```

2.6 Convert a Set to an Array

A **Set** stores only unique values. You can spread it into an array to get a list of unique items.

```
const nums = [1, 2, 2, 3, 3, 3, 4];

const unique = [...new Set(nums)];

console.log(unique); // [1, 2, 3, 4]
```

■ *This is a very popular trick to remove duplicates from an array!*

3. Spread with Objects

3.1 Copy an Object

Just like arrays, spread creates a **shallow copy** of an object.

```
const user = { name: 'Arjun', age: 25 };

const userCopy = { ...user };

userCopy.age = 30;

console.log(user.age); // 25 – original not changed
console.log(userCopy.age); // 30
```

3.2 Merge Two Objects

You can combine properties from multiple objects into one.

```
const basicInfo = { name: 'Priya', age: 22 };

const jobInfo = { role: 'Developer', company: 'TechCo' };

const fullProfile = { ...basicInfo, ...jobInfo };

console.log(fullProfile);

// { name: 'Priya', age: 22, role: 'Developer', company: 'TechCo' }
```

3.3 Override / Update Properties

When two objects have the **same key**, the later one wins. This is great for updating just one field.

```
const defaults = { theme: 'light', fontSize: 14, lang: 'en' };

const userPrefs = { theme: 'dark' };
```

```
const settings = { ...defaults, ...userPrefs };

console.log(settings);

// { theme: 'dark', fontSize: 14, lang: 'en' }

//   ^ overridden      ^ kept          ^ kept
```

■ *Order matters! Properties on the RIGHT side overwrite those on the left when keys are the same.*

3.4 Add New Properties While Spreading

You can add new fields to an object while copying it.

```
const product = { id: 101, name: 'Shoe' };

const updated = { ...product, price: 999, inStock: true };

console.log(updated);

// { id: 101, name: 'Shoe', price: 999, inStock: true }
```

3.5 Remove a Property (Using Destructuring + Spread)

A clever trick: destructure the property you want to remove, then spread the rest.

```
const user = { name: 'Riya', password: 'secret123', age: 28 };

const { password, ...safeUser } = user;

console.log(safeUser);

// { name: 'Riya', age: 28 } — password removed!
```

■ *This pattern is very useful when sending user data to a frontend — you never want to expose the password!*

4. Spread in Function Calls

4.1 Passing Multiple Arguments

When a function needs separate arguments but you have an array, spread them in.

```
function add(a, b, c) {

  return a + b + c;

}
```

```
const nums = [10, 20, 30];

console.log(add(...nums)); // 60
```

4.2 Mixing Spread with Other Arguments

```
function greet(greeting, firstName, lastName) {
  return `${greeting}, ${firstName} ${lastName}!`;
}

const name = ['John', 'Doe'];

console.log(greet('Hello', ...name));

// Hello, John Doe!
```

5. Real-World Examples

5.1 Update State in React (Immutable Update)

In React, you should never change state directly. Spread lets you create a new object with just one field changed.

```
const [user, setUser] = useState({ name: 'Alice', age: 25 });

// Update only the age, keep the rest:

setUser({ ...user, age: 26 });

// Result: { name: 'Alice', age: 26 }
```

5.2 Add an Item to an Array (Immutably)

```
const todos = ['Buy milk', 'Walk dog'];

const newTodos = [...todos, 'Read book'];

console.log(newTodos);

// ['Buy milk', 'Walk dog', 'Read book']

// Original is untouched:

console.log(todos); // ['Buy milk', 'Walk dog']
```

5.3 Merge Default Config with User Config

```
const defaultConfig = {  
  host: 'localhost',  
  port: 3000,  
  debug: false,  
  timeout: 5000  
};  
  
const userConfig = {  
  port: 8080,  
  debug: true  
};  
  
const finalConfig = { ...defaultConfig, ...userConfig };  
  
console.log(finalConfig);  
  
// { host: 'localhost', port: 8080, debug: true, timeout: 5000 }
```

5.4 Flatten One Level of Nested Arrays

```
const groups = [[1, 2], [3, 4], [5, 6]];  
  
const flat = [].concat(...groups);  
  
console.log(flat); // [1, 2, 3, 4, 5, 6]
```

■ For deep nesting, use `arr.flat()` or `arr.flat(Infinity)` — but for one level, `spread + concat` works perfectly.

5.5 Clone and Modify in One Step

```
const order = { id: 1, item: 'Pizza', status: 'pending' };  
  
// Create a new order object with updated status:  
const delivered = { ...order, status: 'delivered' };  
  
console.log(order.status); // pending  
console.log(delivered.status); // delivered
```

6. Spread vs Rest — What is the Difference?

Both use ... but they do opposite things depending on where you use them.

Feature	Spread (...)	Rest (...)
What it does	Expands values outward	Collects values inward
Used in	Function calls, array/object literals	Function params, destructuring
Direction	One thing → Many pieces	Many pieces → One thing
Example	<code>Math.max(...arr)</code>	<code>function f(...args)</code>
Result	Spreads items apart	Groups items together

7. Quick Reference Table

Use Case	Example Code	Result
Copy array	<code>[...arr]</code>	New array copy
Merge arrays	<code>[...a, ...b]</code>	Combined array
Add items to array	<code>[1, ...arr, 5]</code>	Items inserted
Pass array as args	<code>fn(...arr)</code>	Each item as arg
String to char array	<code>[...'Hello']</code>	<code>['H','e','l','l','o']</code>
Remove duplicates	<code>[...new Set(arr)]</code>	Unique values only
Copy object	<code>{ ...obj }</code>	New object copy
Merge objects	<code>{ ...a, ...b }</code>	Combined object
Override one property	<code>{ ...obj, key: val }</code>	Updated copy
Remove property	<code>const { key, ...rest } = obj</code>	obj without key

8. Common Mistakes to Avoid

- **Shallow copy only:** Spread makes a *shallow* copy. If an object has nested objects inside, those inner objects are still shared — not deeply copied.
- **Spreading non-iterables:** You cannot spread a plain number or boolean. Only arrays, strings, objects, Sets, and Maps can be spread.
- **Order matters in objects:** When merging objects, the right-most value wins for duplicate keys. Always put defaults first, overrides last.
- **Confusing spread and rest:** Spread is used when *calling* a function or creating an array/object. Rest is used in function *definitions* or destructuring.
- **Spreading undefined/null:** Spreading `undefined` or `null` into an object is safe (it is ignored), but spreading them into an array throws an error.

9. Shallow Copy — Important Detail

Spread only copies the **top level** of an object or array. Nested objects are still referenced, not copied.

```
const original = { name: 'Tom', address: { city: 'Delhi' } };

const copy = { ...original };

copy.name = 'Jerry';           // OK – top level, independent
copy.address.city = 'Mumbai'; // PROBLEM – same reference!

console.log(original.name);     // Tom    (unchanged)
console.log(original.address.city); // Mumbai (changed!)
```

■ To deep-copy an object, use `structuredClone(obj)` in modern JS, or `JSON.parse(JSON.stringify(obj))` for simple objects without functions.

10. Pro Tips

- Use `[...new Set(arr)]` as a one-liner to remove duplicate values from any array.
- In React and Redux, always use spread to update state immutably — never mutate state directly.
- Spread works with any **iterable**: arrays, strings, Sets, Maps, generator functions, NodeLists.
- Combine spread with `Object.entries()` and `Object.fromEntries()` for powerful object transformations.
- Use `{ ...defaults, ...userOptions }` as a clean pattern for function option merging.
- Spread inside `console.log(...arr)` prints all items on one line — handy for debugging.